

▼ Data Manipulation and Visualization Examples

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'Grace', 'Heidi', 'Ivan', 'Judy'],
    'Age': [25, 30, 35, 28, 40, 22, 32, 29, 38, 27],
    'City': ['New York', 'Los Angeles', 'Chicago', 'New York', 'Los Angeles', 'Chicago', 'New York', 'Boston', 'Seattle', 'Miami'],
    'Score': [85, 92, 78, 88, 95, 70, 81, 90, 75, 83],
    'Gender': ['Female', 'Male', 'Male', 'Male', 'Female', 'Male', 'Female', 'Female', 'Male', 'Female'],
    'Salary': [60000, 75000, 80000, 62000, 90000, 55000, 70000, 68000, 82000, 63000],
    'Experience': [2, 7, 10, 4, 15, 1, 6, 5, 12, 3],
    'Rating': [4.2, np.nan, 3.9, 4.5, 4.8, 3.5, 4.1, np.nan, 3.7, 4.3]
}
df = pd.DataFrame(data)

print("Sample DataFrame created:")
display(df)
```

Sample DataFrame created:

	Name	Age	City	Score	Gender	Salary	Experience	Rating
0	Alice	25	New York	85	Female	60000	2	4.2
1	Bob	30	Los Angeles	92	Male	75000	7	NaN
2	Charlie	35	Chicago	78	Male	80000	10	3.9
3	David	28	New York	88	Male	62000	4	4.5
4	Eve	40	Los Angeles	95	Female	90000	15	4.8
5	Frank	22	Chicago	70	Male	55000	1	3.5
6	Grace	32	New York	81	Female	70000	6	4.1
7	Heidi	29	Boston	90	Female	68000	5	NaN
8	Ivan	38	Seattle	75	Male	82000	12	3.7
9	Judy	27	Miami	83	Female	63000	3	4.3

▼ Filtering Data

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Create a sample DataFrame for this individual cell's execution
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'Grace', 'Heidi', 'Ivan', 'Judy'],
    'Age': [25, 30, 35, 28, 40, 22, 32, 29, 38, 27],
    'City': ['New York', 'Los Angeles', 'Chicago', 'New York', 'Los Angeles', 'Chicago', 'New York', 'Boston', 'Seattle', 'Miami'],
    'Score': [85, 92, 78, 88, 95, 70, 81, 90, 75, 83],
    'Gender': ['Female', 'Male', 'Male', 'Male', 'Female', 'Male', 'Female', 'Female', 'Male', 'Female'],
    'Salary': [60000, 75000, 80000, 62000, 90000, 55000, 70000, 68000, 82000, 63000],
    'Experience': [2, 7, 10, 4, 15, 1, 6, 5, 12, 3],
    'Rating': [4.2, np.nan, 3.9, 4.5, 4.8, 3.5, 4.1, np.nan, 3.7, 4.3]
}
df = pd.DataFrame(data)

filtered_df_age = df[df['Age'] > 30]
print("DataFrame filtered by Age > 30:")
display(filtered_df_age)
```

DataFrame filtered by Age > 30:

	Name	Age	City	Score	Gender	Salary	Experience	Rating
2	Charlie	35	Chicago	78	Male	80000	10	3.9
4	Eve	40	Los Angeles	95	Female	90000	15	4.8
6	Grace	32	New York	81	Female	70000	6	4.1
8	Ivan	38	Seattle	75	Male	82000	12	3.7

Filtering Data with Multiple Conditions

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'Grace', 'Heidi', 'Ivan', 'Judy'],
    'Age': [25, 30, 35, 28, 40, 22, 32, 29, 38, 27],
    'City': ['New York', 'Los Angeles', 'Chicago', 'New York', 'Los Angeles', 'Chicago', 'New York', 'Boston', 'Seattle',
    'Score': [85, 92, 78, 88, 95, 70, 81, 90, 75, 83],
    'Gender': ['Female', 'Male', 'Male', 'Male', 'Female', 'Male', 'Female', 'Female', 'Male', 'Female'],
    'Salary': [60000, 75000, 80000, 62000, 90000, 55000, 70000, 68000, 82000, 63000],
    'Experience': [2, 7, 10, 4, 15, 1, 6, 5, 12, 3],
    'Rating': [4.2, np.nan, 3.9, 4.5, 4.8, 3.5, 4.1, np.nan, 3.7, 4.3]
}
df = pd.DataFrame(data)

filtered_df_multiple = df[(df['City'] == 'New York') & (df['Score'] > 80)]
print("DataFrame filtered by City == 'New York' AND Score > 80:")
display(filtered_df_multiple)
```

DataFrame filtered by City == 'New York' AND Score > 80:

	Name	Age	City	Score	Gender	Salary	Experience	Rating
0	Alice	25	New York	85	Female	60000	2	4.2
3	David	28	New York	88	Male	62000	4	4.5
6	Grace	32	New York	81	Female	70000	6	4.1

Sorting Data (Ascending Order)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Create a sample DataFrame for this individual cell's execution
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'Grace', 'Heidi', 'Ivan', 'Judy'],
    'Age': [25, 30, 35, 28, 40, 22, 32, 29, 38, 27],
    'City': ['New York', 'Los Angeles', 'Chicago', 'New York', 'Los Angeles', 'Chicago', 'New York', 'Boston', 'Seattle',
    'Score': [85, 92, 78, 88, 95, 70, 81, 90, 75, 83],
    'Gender': ['Female', 'Male', 'Male', 'Male', 'Female', 'Male', 'Female', 'Female', 'Male', 'Female'],
    'Salary': [60000, 75000, 80000, 62000, 90000, 55000, 70000, 68000, 82000, 63000],
    'Experience': [2, 7, 10, 4, 15, 1, 6, 5, 12, 3],
    'Rating': [4.2, np.nan, 3.9, 4.5, 4.8, 3.5, 4.1, np.nan, 3.7, 4.3]
}
df = pd.DataFrame(data)

sorted_df_asc = df.sort_values(by='Salary', ascending=True)
print("DataFrame sorted by Salary (Ascending):")
display(sorted_df_asc)
```

DataFrame sorted by Salary (Ascending):

	Name	Age	City	Score	Gender	Salary	Experience	Rating
5	Frank	22	Chicago	70	Male	55000	1	3.5
0	Alice	25	New York	85	Female	60000	2	4.2
3	David	28	New York	88	Male	62000	4	4.5
9	Judy	27	Miami	83	Female	63000	3	4.3
7	Heidi	29	Boston	90	Female	68000	5	NaN
6	Grace	32	New York	81	Female	70000	6	4.1
1	Bob	30	Los Angeles	92	Male	75000	7	NaN
2	Charlie	35	Chicago	78	Male	80000	10	3.9
8	Ivan	38	Seattle	75	Male	82000	12	3.7
4	Eve	40	Los Angeles	95	Female	90000	15	4.8

Sorting Data (Descending Order)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Create a sample DataFrame for this individual cell's execution
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'Grace', 'Heidi', 'Ivan', 'Judy'],
    'Age': [25, 30, 35, 28, 40, 22, 32, 29, 38, 27],
    'City': ['New York', 'Los Angeles', 'Chicago', 'New York', 'Los Angeles', 'Chicago', 'New York', 'Boston', 'Seattle', 'New York'],
    'Score': [85, 92, 78, 88, 95, 70, 81, 90, 75, 83],
    'Gender': ['Female', 'Male', 'Male', 'Male', 'Female', 'Male', 'Female', 'Female', 'Male', 'Female'],
    'Salary': [60000, 75000, 80000, 62000, 90000, 55000, 70000, 68000, 82000, 63000],
    'Experience': [2, 7, 10, 4, 15, 1, 6, 5, 12, 3],
    'Rating': [4.2, np.nan, 3.9, 4.5, 4.8, 3.5, 4.1, np.nan, 3.7, 4.3]
}
df = pd.DataFrame(data)

sorted_df_desc = df.sort_values(by='Score', ascending=False)
print("DataFrame sorted by Score (Descending):")
display(sorted_df_desc)
```

DataFrame sorted by Score (Descending):

	Name	Age	City	Score	Gender	Salary	Experience	Rating
4	Eve	40	Los Angeles	95	Female	90000	15	4.8
1	Bob	30	Los Angeles	92	Male	75000	7	NaN
7	Heidi	29	Boston	90	Female	68000	5	NaN
3	David	28	New York	88	Male	62000	4	4.5
0	Alice	25	New York	85	Female	60000	2	4.2
9	Judy	27	Miami	83	Female	63000	3	4.3
6	Grace	32	New York	81	Female	70000	6	4.1
2	Charlie	35	Chicago	78	Male	80000	10	3.9
8	Ivan	38	Seattle	75	Male	82000	12	3.7
5	Frank	22	Chicago	70	Male	55000	1	3.5

Handling Missing Values: Detecting and Counting

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Create a sample DataFrame for this individual cell's execution
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'Grace', 'Heidi', 'Ivan', 'Judy'],
    'Age': [25, 30, 35, 28, 40, 22, 32, 29, 38, 27],
    'City': ['New York', 'Los Angeles', 'Chicago', 'New York', 'Los Angeles', 'Chicago', 'New York', 'Boston', 'Seattle', 'New York'],
    'Score': [85, 92, 78, 88, 95, 70, 81, 90, 75, 83],
    'Gender': ['Female', 'Male', 'Male', 'Male', 'Female', 'Male', 'Female', 'Female', 'Male', 'Female'],
    'Salary': [60000, 75000, 80000, 62000, 90000, 55000, 70000, 68000, 82000, 63000],
    'Experience': [2, 7, 10, 4, 15, 1, 6, 5, 12, 3],
    'Rating': [4.2, np.nan, 3.9, 4.5, 4.8, 3.5, 4.1, np.nan, 3.7, 4.3]
}
df = pd.DataFrame(data)

print("DataFrame with boolean indicators for missing values:")
display(df.isnull().head())

missing_counts = df.isnull().sum()
print("\nNumber of missing values per column:")
display(missing_counts)
```

DataFrame with boolean indicators for missing values:

	Name	Age	City	Score	Gender	Salary	Experience	Rating
0	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	True
2	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False

Number of missing values per column:

	0
Name	0
Age	0
City	0
Score	0
Gender	0
Salary	0
Experience	0
Rating	2

dtype: int64

Handling Missing Values: Filling Missing Values (Example)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'Grace', 'Heidi', 'Ivan', 'Judy'],
    'Age': [25, 30, 35, 28, 40, 22, 32, 29, 38, 27],
    'City': ['New York', 'Los Angeles', 'Chicago', 'New York', 'Los Angeles', 'Chicago', 'New York', 'Boston', 'Seattle', 'M'],
    'Score': [85, 92, 78, 88, 95, 70, 81, 90, 75, 83],
    'Gender': ['Female', 'Male', 'Male', 'Male', 'Female', 'Male', 'Female', 'Female', 'Male', 'Female'],
    'Salary': [60000, 75000, 80000, 62000, 90000, 55000, 70000, 68000, 82000, 63000],
    'Experience': [2, 7, 10, 4, 15, 1, 6, 5, 12, 3],
    'Rating': [4.2, np.nan, 3.9, 4.5, 4.8, 3.5, 4.1, np.nan, 3.7, 4.3]
}
df = pd.DataFrame(data)

df_filled = df.copy()
df_filled['Rating'].fillna(df_filled['Rating'].mean(), inplace=True)
print("DataFrame after filling missing 'Rating' values with mean:")
display(df_filled)
```

DataFrame after filling missing 'Rating' values with mean:

/tmp/ipykernel_765/832200291.py:20: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through c[...]
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]

```
df_filled['Rating'].fillna(df_filled['Rating'].mean(), inplace=True)
```

	Name	Age	City	Score	Gender	Salary	Experience	Rating
0	Alice	25	New York	85	Female	60000	2	4.200
1	Bob	30	Los Angeles	92	Male	75000	7	4.125
2	Charlie	35	Chicago	78	Male	80000	10	3.900
3	David	28	New York	88	Male	62000	4	4.500
4	Eve	40	Los Angeles	95	Female	90000	15	4.800
5	Frank	22	Chicago	70	Male	55000	1	3.500
6	Grace	32	New York	81	Female	70000	6	4.100
7	Heidi	29	Boston	90	Female	68000	5	4.125
8	Ivan	38	Seattle	75	Male	82000	12	3.700
9	Judy	27	Miami	83	Female	63000	3	4.300

✓ Removing Duplicate Data

```
import pandas as pd

duplicate_data = {
    'ID': [1, 2, 3, 1, 4, 5],
    'Value': ['A', 'B', 'C', 'A', 'D', 'E']
}
df_duplicates = pd.DataFrame(duplicate_data)
print("Original DataFrame with duplicates:")
display(df_duplicates)

# Remove duplicate rows based on all columns
df_no_duplicates = df_duplicates.drop_duplicates()
print("\nDataFrame after removing duplicates:")
display(df_no_duplicates)
```

Original DataFrame with duplicates:

	ID	Value
0	1	A
1	2	B
2	3	C
3	1	A
4	4	D
5	5	E

DataFrame after removing duplicates:

	ID	Value
0	1	A
1	2	B
2	3	C
4	4	D
5	5	E

✓ Data Visualization: Line Plot

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

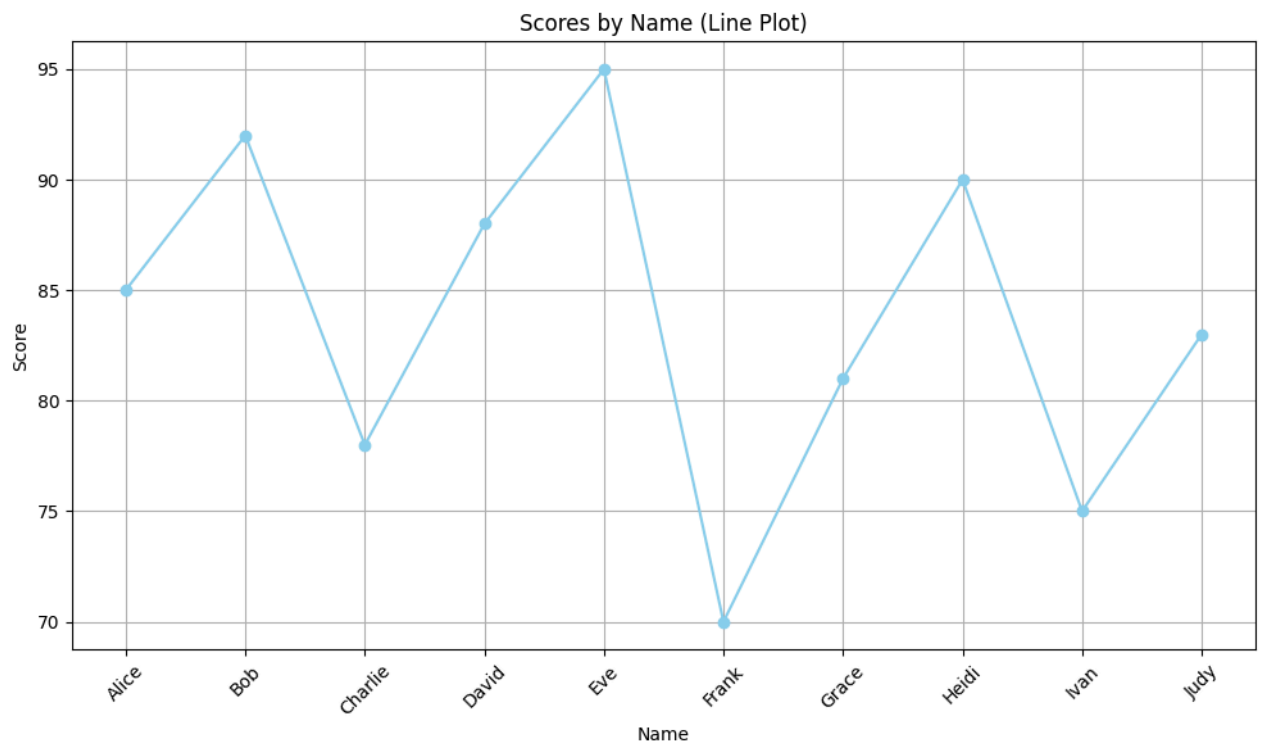
# Create a sample DataFrame for this individual cell's execution
```

```

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'Grace', 'Heidi', 'Ivan', 'Judy'],
    'Age': [25, 30, 35, 28, 40, 22, 32, 29, 38, 27],
    'City': ['New York', 'Los Angeles', 'Chicago', 'New York', 'Los Angeles', 'Chicago', 'New York', 'Boston', 'Seattle', 'New York'],
    'Score': [85, 92, 78, 88, 95, 70, 81, 90, 75, 83],
    'Gender': ['Female', 'Male', 'Male', 'Male', 'Female', 'Male', 'Female', 'Female', 'Male', 'Female'],
    'Salary': [60000, 75000, 80000, 62000, 90000, 55000, 70000, 68000, 82000, 63000],
    'Experience': [2, 7, 10, 4, 15, 1, 6, 5, 12, 3],
    'Rating': [4.2, np.nan, 3.9, 4.5, 4.8, 3.5, 4.1, np.nan, 3.7, 4.3]
}
df = pd.DataFrame(data)

plt.figure(figsize=(10, 6))
plt.plot(df['Name'], df['Score'], marker='o', linestyle='-', color='skyblue')
plt.xlabel('Name')
plt.ylabel('Score')
plt.title('Scores by Name (Line Plot)')
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```



▼ Data Visualization: Bar Chart

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Create a sample DataFrame for this individual cell's execution
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'Grace', 'Heidi', 'Ivan', 'Judy'],
    'Age': [25, 30, 35, 28, 40, 22, 32, 29, 38, 27],
    'City': ['New York', 'Los Angeles', 'Chicago', 'New York', 'Los Angeles', 'Chicago', 'New York', 'Boston', 'Seattle', 'New York'],
    'Score': [85, 92, 78, 88, 95, 70, 81, 90, 75, 83],
    'Gender': ['Female', 'Male', 'Male', 'Male', 'Female', 'Male', 'Female', 'Female', 'Male', 'Female'],
    'Salary': [60000, 75000, 80000, 62000, 90000, 55000, 70000, 68000, 82000, 63000],
    'Experience': [2, 7, 10, 4, 15, 1, 6, 5, 12, 3],
    'Rating': [4.2, np.nan, 3.9, 4.5, 4.8, 3.5, 4.1, np.nan, 3.7, 4.3]
}
df = pd.DataFrame(data)

plt.figure(figsize=(10, 6))
sns.barplot(x='City', y='Salary', data=df, palette='viridis')
plt.xlabel('City')
plt.ylabel('Salary')
plt.title('Average Salary by City (Bar Chart)')
plt.xticks(rotation=45)

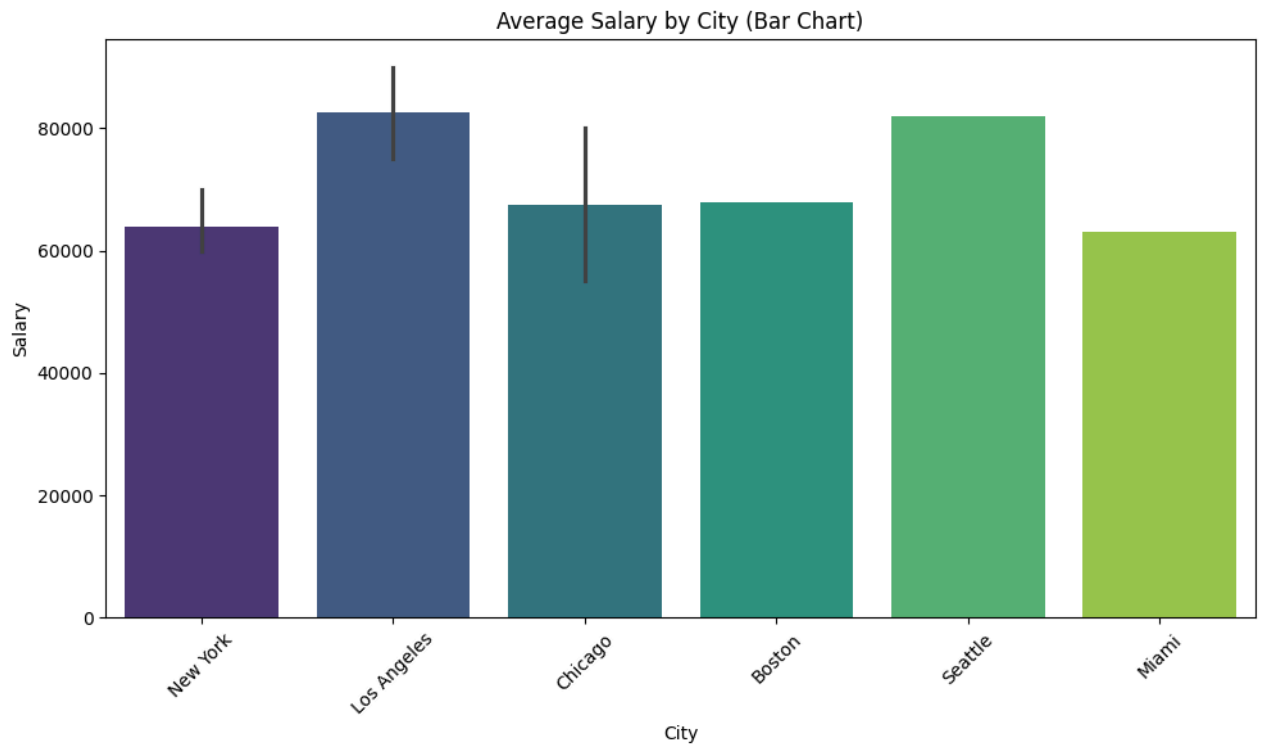
```

```
plt.tight_layout()
plt.show()
```

/tmp/ipykernel_765/2632723525.py:20: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

```
sns.barplot(x='City', y='Salary', data=df, palette='viridis')
```

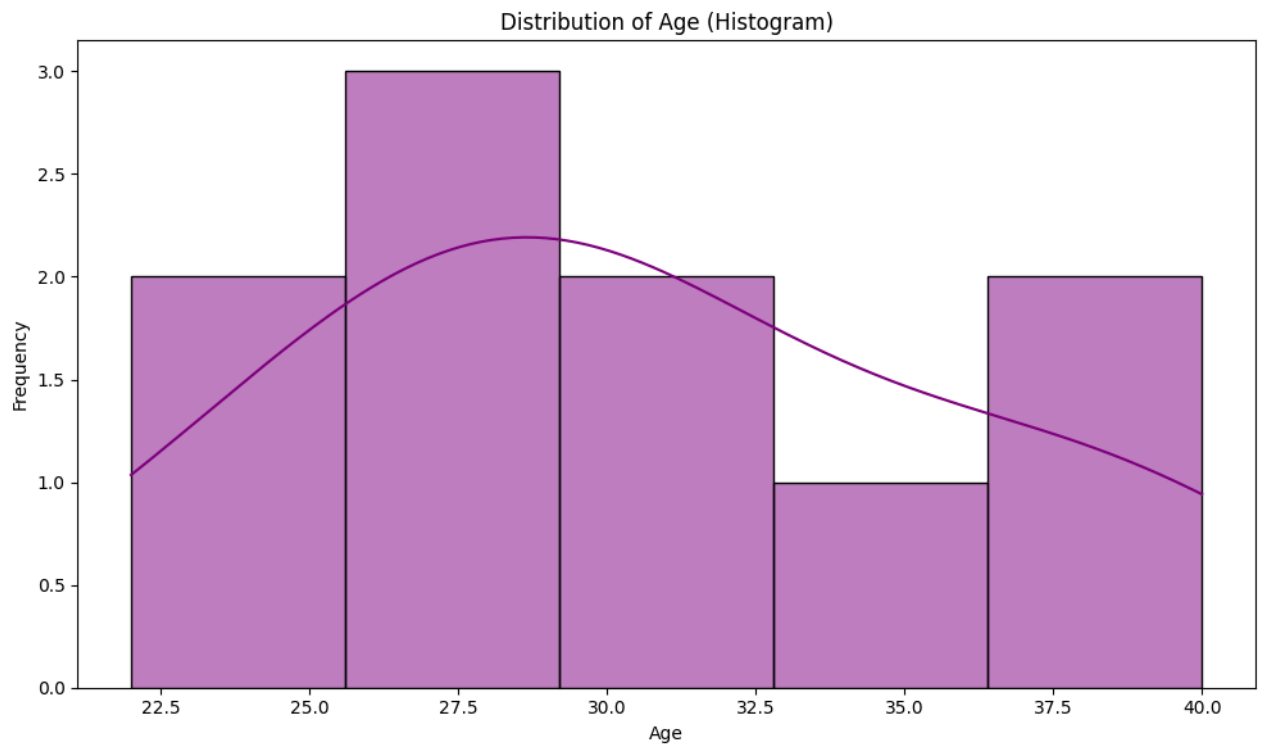


▼ Data Visualization: Histogram

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Create a sample DataFrame for this individual cell's execution
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'Grace', 'Heidi', 'Ivan', 'Judy'],
    'Age': [25, 30, 35, 28, 40, 22, 32, 29, 38, 27],
    'City': ['New York', 'Los Angeles', 'Chicago', 'New York', 'Los Angeles', 'Chicago', 'New York', 'Boston', 'Seattle', 'M'],
    'Score': [85, 92, 78, 88, 95, 70, 81, 90, 75, 83],
    'Gender': ['Female', 'Male', 'Male', 'Male', 'Female', 'Male', 'Female', 'Female', 'Male', 'Female'],
    'Salary': [60000, 75000, 80000, 62000, 90000, 55000, 70000, 68000, 82000, 63000],
    'Experience': [2, 7, 10, 4, 15, 1, 6, 5, 12, 3],
    'Rating': [4.2, np.nan, 3.9, 4.5, 4.8, 3.5, 4.1, np.nan, 3.7, 4.3]
}
df = pd.DataFrame(data)

plt.figure(figsize=(10, 6))
sns.histplot(df['Age'], bins=5, kde=True, color='purple')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.title('Distribution of Age (Histogram)')
plt.tight_layout()
plt.show()
```



▼ Data Visualization: Scatter Plot

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Create a sample DataFrame for this individual cell's execution
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'Grace', 'Heidi', 'Ivan', 'Judy'],
    'Age': [25, 30, 35, 28, 40, 22, 32, 29, 38, 27],
    'City': ['New York', 'Los Angeles', 'Chicago', 'New York', 'Los Angeles', 'Chicago', 'New York', 'Boston', 'Seattle', 'New York'],
    'Score': [85, 92, 78, 88, 95, 70, 81, 90, 75, 83],
    'Gender': ['Female', 'Male', 'Male', 'Male', 'Female', 'Male', 'Female', 'Female', 'Male', 'Female'],
    'Salary': [60000, 75000, 80000, 62000, 90000, 55000, 70000, 68000, 82000, 63000],
    'Experience': [2, 7, 10, 4, 15, 1, 6, 5, 12, 3],
    'Rating': [4.2, np.nan, 3.9, 4.5, 4.8, 3.5, 4.1, np.nan, 3.7, 4.3]
}
df = pd.DataFrame(data)

plt.figure(figsize=(10, 6))
sns.scatterplot(x='Experience', y='Salary', hue='Gender', data=df, s=100, alpha=0.8)
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.title('Salary vs. Experience (Scatter Plot)')
plt.grid(True)
plt.tight_layout()
plt.show()
```




▼ Data Visualization: Pie Chart

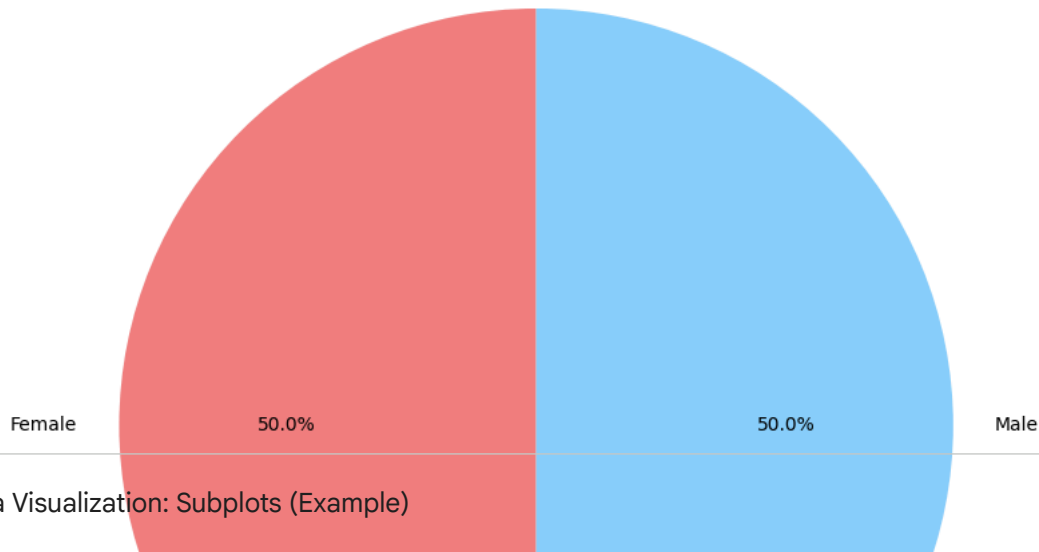
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'Grace', 'Heidi', 'Ivan', 'Judy'],
    'Age': [25, 30, 35, 28, 40, 22, 32, 29, 38, 27],
    'City': ['New York', 'Los Angeles', 'Chicago', 'New York', 'Los Angeles', 'Chicago', 'New York', 'Boston', 'Seattle', 'New York'],
    'Score': [85, 92, 78, 88, 95, 70, 81, 90, 75, 83],
    'Gender': ['Female', 'Male', 'Male', 'Male', 'Female', 'Male', 'Female', 'Female', 'Male', 'Female'],
    'Salary': [60000, 75000, 80000, 62000, 90000, 55000, 70000, 68000, 82000, 63000],
    'Experience': [2, 7, 10, 4, 15, 1, 6, 5, 12, 3],
    'Rating': [4.2, np.nan, 3.9, 4.5, 4.8, 3.5, 4.1, np.nan, 3.7, 4.3]
}
df = pd.DataFrame(data)

gender_counts = df['Gender'].value_counts()

plt.figure(figsize=(8, 8))
plt.pie(gender_counts, labels=gender_counts.index, autopct='%1.1f%%', startangle=90, colors=['lightcoral', 'lightskyblue'])
plt.title('Distribution of Gender (Pie Chart)')
plt.axis('equal')
plt.tight_layout()
plt.show()
```

Distribution of Gender (Pie Chart)



▼ Data Visualization: Subplots (Example)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Create a sample DataFrame for this individual cell's execution
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'Grace', 'Heidi', 'Ivan', 'Judy'],
    'Age': [25, 30, 35, 28, 40, 22, 32, 29, 38, 27],
    'City': ['New York', 'Los Angeles', 'Chicago', 'New York', 'Los Angeles', 'Chicago', 'New York', 'Boston', 'Seattle', 'New York'],
    'Score': [85, 92, 78, 88, 95, 70, 81, 90, 75, 83],
    'Gender': ['Female', 'Male', 'Male', 'Male', 'Female', 'Male', 'Female', 'Female', 'Male', 'Female'],
    'Salary': [60000, 75000, 80000, 62000, 90000, 55000, 70000, 68000, 82000, 63000],
    'Experience': [2, 7, 10, 4, 15, 1, 6, 5, 12, 3],
    'Rating': [4.2, np.nan, 3.9, 4.5, 4.8, 3.5, 4.1, np.nan, 3.7, 4.3]
}
df = pd.DataFrame(data)

fig, axes = plt.subplots(1, 2, figsize=(15, 6))
```